

# Лабораторная работа №1 | Разработка моделей

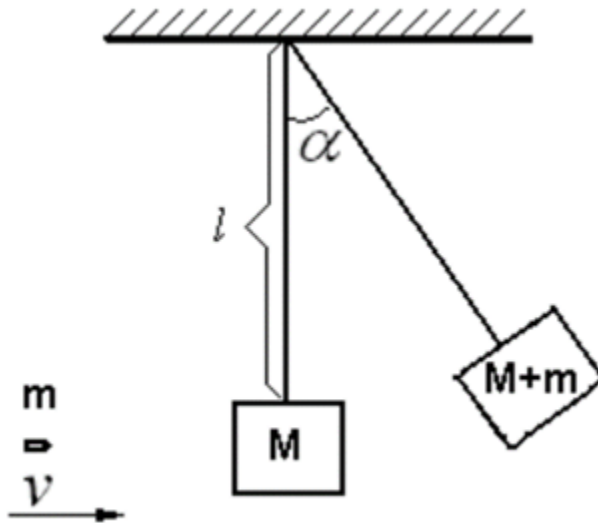
Цели работы:

- Изучение основ построения моделей в Python с использованием NumPy и Matplotlib
- Изучение приёмов построения моделей с графическим пользовательским интерфейсом (GUI) на основе Tkinter или PyQt/PySide
- Получение практических навыков разработки моделей с вычислениями, визуализацией и интерактивными приложениями

## Теоретическая часть

### Пример 1. Разработка модели определения скорости пули.

Система состоит из груза, подвешенного на нити, и пули, которая попадает в груз. После удара груз с пулей отклоняется на угол  $\alpha$ .



## Законы сохранения

Для описания системы воспользуемся законами сохранения

### *Закон сохранения энергии*

$$\frac{(M + m) V^2}{2} = (M + m) \cdot g \cdot l (1 - \cos \alpha) \quad (1)$$

где:

- $m$  — масса пули (кг);
- $M$  — масса груза (кг);
- $v$  — скорость пули (м/с);
- $V$  — скорость системы «пуля-груз» (м/с);
- $l$  — длина нити подвеса (м);
- $g$  — ускорение свободного падения (м/с<sup>2</sup>);
- $\alpha$  — угол отклонения подвеса от вертикали (рад).

### *Закон сохранения импульса*

$$m \cdot v = (M + m) \cdot V \quad (2)$$

Из уравнения (2) находим начальную скорость системы «пуля-груз»:

$$V = \frac{m \cdot v}{M + m} \quad (3)$$

## Вычисление искомых параметров

### **Рассмотрим два варианта:**

- а) при известной скорости пули определить угол отклонения груза;
- б) по известному углу отклонения груза определить скорость пули.

Подставляя выражение (3) в (1) находим интересующие нас параметры.

$$\text{а) } \alpha = \arccos \left( 1 - \frac{m^2 v^2}{2 (M + m)^2 g l} \right)$$

$$\text{б) } v = \sqrt{\frac{2 (M + m)^2 g l (1 - \cos \alpha)}{m^2}}$$

или эквивалентно:

$$v = \frac{M + m}{m} \sqrt{2 g l (1 - \cos \alpha)}$$

# Средства языка Python

## Библиотека NumPy

Для численных расчётов в Python используется библиотека **NumPy**. Она позволяет:

- работать с векторами и матрицами как с едиными объектами;
- выполнять операции сразу над массивами (без явных циклов);
- использовать большое количество встроенных математических функций.

Чтобы начать работать с NumPy, его нужно импортировать:

```
In [1]: import numpy as np
```

Пример простых операций:

```
In [2]: x = np.linspace(0, 1, 11) # 11 точек от 0 до 1
        y = np.cos(x) # косинус для всего массива x
```

## Библиотека Matplotlib

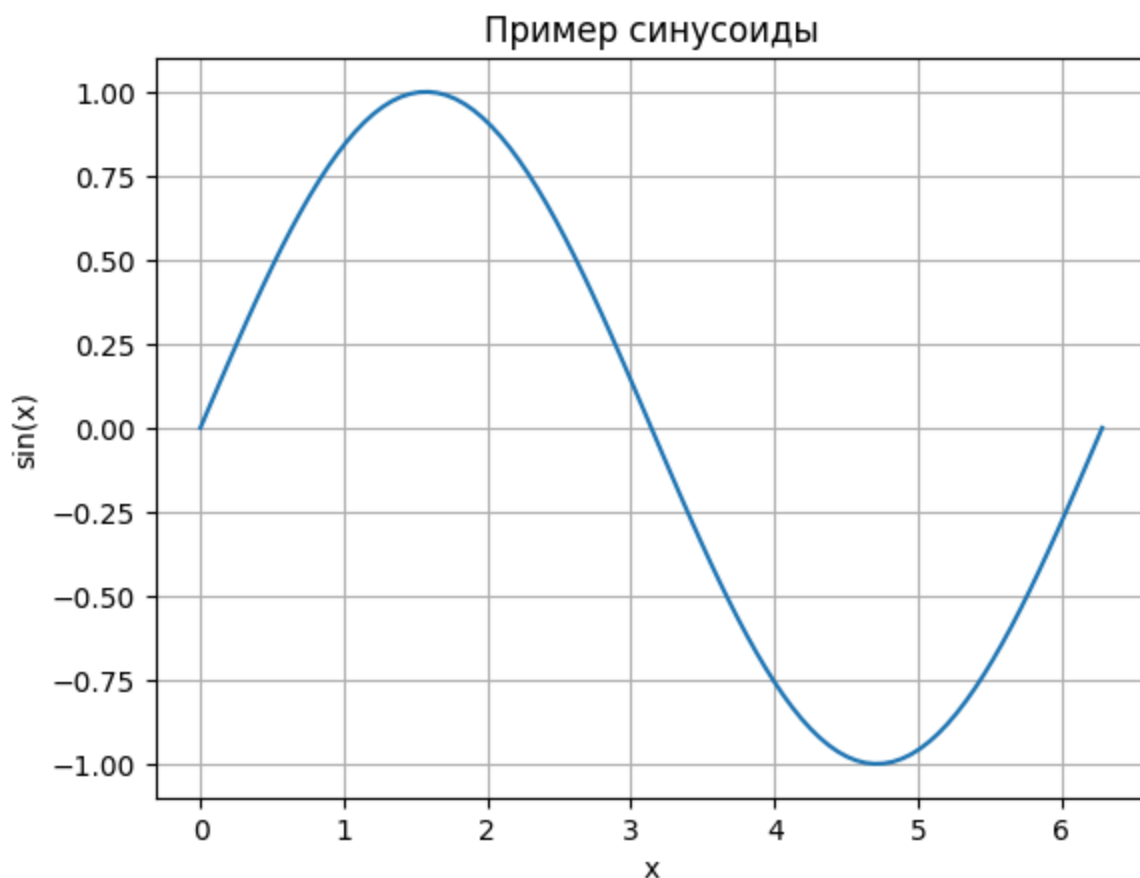
Для построения графиков в Python используется библиотека **Matplotlib**, модуль `pyplot`. [web:11]

Базовый шаблон:

```
In [3]: import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

plt.plot(x, y)
plt.grid(True)
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.title('Пример синусоиды')
plt.show()
```



## Преобразование градусов и радиан

Во многих вычислительных задачах удобно явно указывать, в каких единицах измеряется угол.

### Градусы → радианы

$$\alpha_{\text{рад}} = \frac{\pi}{180} \cdot \alpha_{\text{град}}$$

### Радианы → градусы

$$\alpha_{\text{град}} = \frac{180}{\pi} \cdot \alpha_{\text{рад}}$$

### Пример кода в Python (используя NumPy):

```
In [4]: import numpy as np

alpha_grad = 45 # градусы
alpha_rad = np.deg2rad(alpha_grad) # в радианы

print(alpha_grad, alpha_rad)

alpha_rad = 0.785 # радианы
```

```
alpha_grad = np.rad2deg(alpha_rad) # в градусы

print(alpha_rad, alpha_grad)
```

```
45 0.7853981633974483
0.785 44.97718691776962
```

## Модель на Python

Модель включает:

- вычисление угла отклонения груза в зависимости от массы пули;
- вычисление скорости пули в зависимости от угла отклонения;
- построение двух графиков в одном окне (`plt.subplots`).

```
In [5]: import numpy as np
import matplotlib.pyplot as plt

# параметры системы
M = 10.0 # масса груза, кг
g = 9.8 # ускорение свободного падения, м/с^2
l = 1.0 # длина подвеса груза, м

# модель влияния массы пули на угол отклонения груза
v = 300.0 # скорость пули, м/с
m = np.arange(0.001, 0.03 + 0.001, 0.001) # масса пули, кг

alpha_rad = np.arccos(1 - m**2 * v**2 / (2 * (m + M)**2 * g * l))
alpha_deg = np.rad2deg(alpha_rad) # радианы → градусы

# создаём окно с двумя подграфиками
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6, 8))

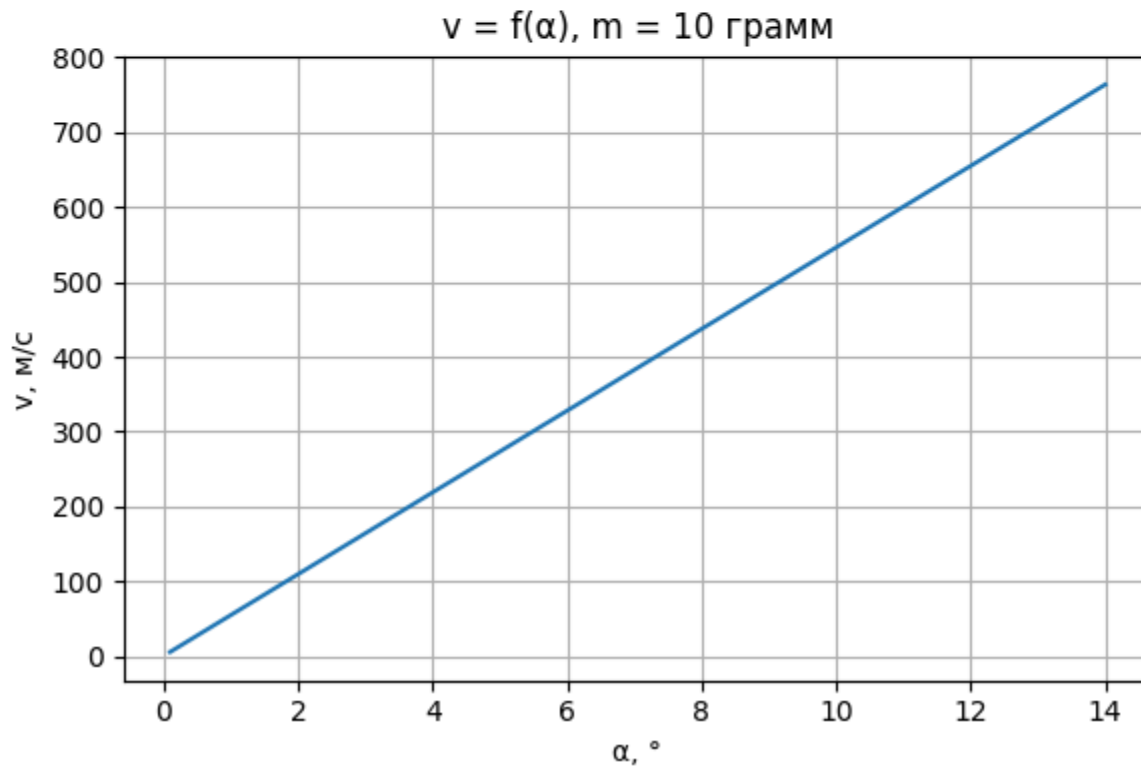
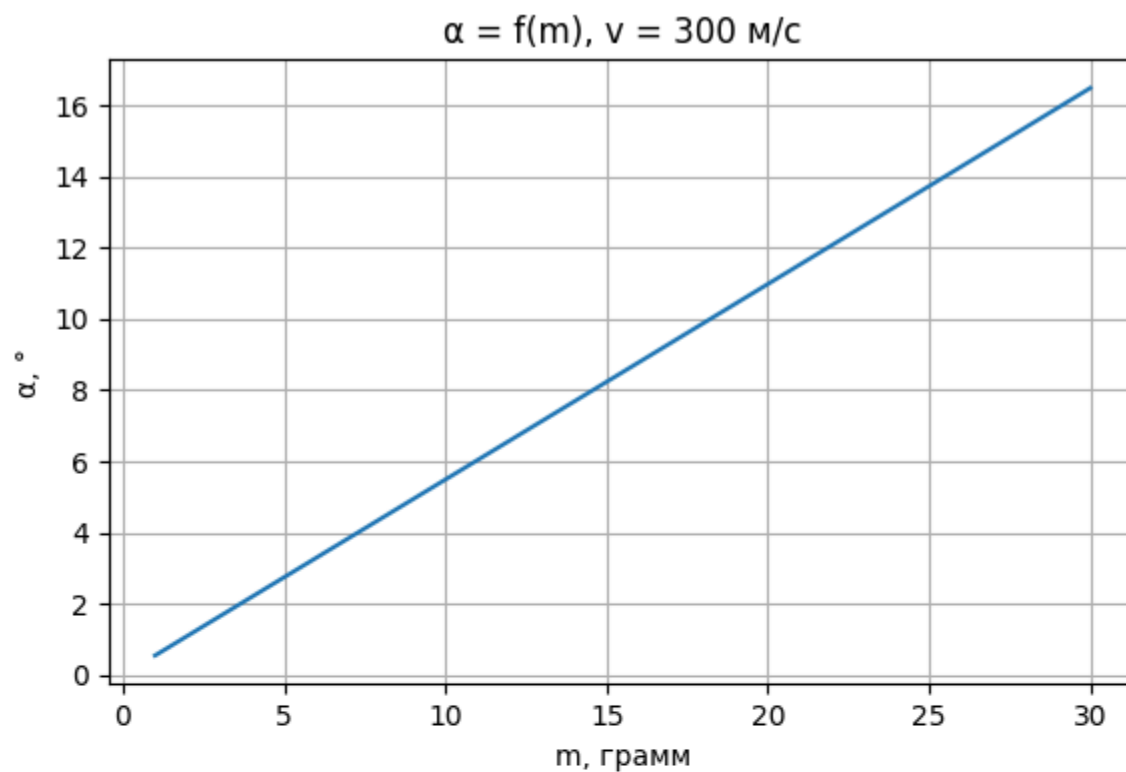
# график зависимости угла от массы пули
ax1.plot(m * 1000, alpha_deg)
ax1.grid(True)
ax1.set_xlabel('m, грамм')
ax1.set_ylabel('α, °')
ax1.set_title('α = f(m), v = 300 м/с')

# модель скорости пули в зависимости от угла отклонения груза
m_const = 0.01 # масса пули, кг
alpha_deg2 = np.arange(0.1, 14.0 + 0.1, 0.1)
alpha_rad2 = np.deg2rad(alpha_deg2)

v2 = np.sqrt(2 * (m_const + M)**2 * g * l * (1 - np.cos(alpha_rad2))) / (m_const + M)

ax2.plot(alpha_deg2, v2)
ax2.grid(True)
ax2.set_xlabel('α, °')
ax2.set_ylabel('v, м/с')
ax2.set_title('v = f(α), m = 10 грамм')
```

```
plt.tight_layout()  
plt.show()
```



# Введение в 3D-графику в Matplotlib

Для построения трёхмерных графиков в Python используется библиотека Matplotlib и её модуль для 3D-оси.

Основные шаги:

- создать фигуру и ось с параметром `projection="3d"` ;
- задать данные (кривую, точки или поверхность);
- вызвать соответствующий метод ( `plot3D` , `scatter3D` , `plot_surface` );
- отобразить результат через `plt.show()` .

## Подготовка: импорт необходимых модулей

```
In [6]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # активация 3D-возможностей
```

## Построение 3D-кривой (траектории)

Ниже приведён пример построения пространственной траектории  $(x(t), y(t), z(t))$

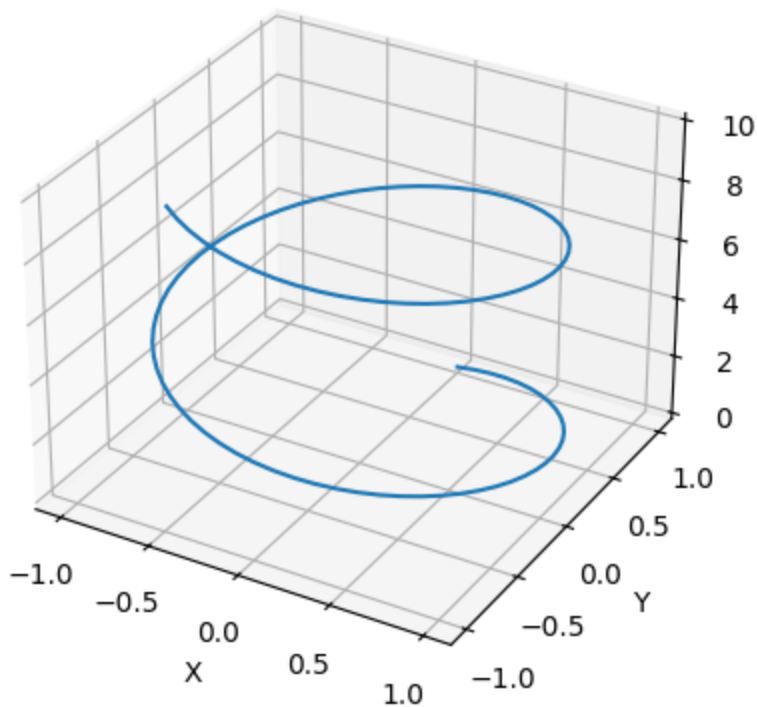
```
In [7]: fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

t = np.linspace(0, 10, 100)
x = np.sin(t)
y = np.cos(t)
z = t

ax.plot3D(x, y, z)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Пример 3D-траектории')

plt.show()
```

## Пример 3D-траектории



## Построение 3D-поверхности

Для визуализации зависимости ( $z = f(x, y)$ ) удобно использовать метод `plot_surface`.

```
In [8]: fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

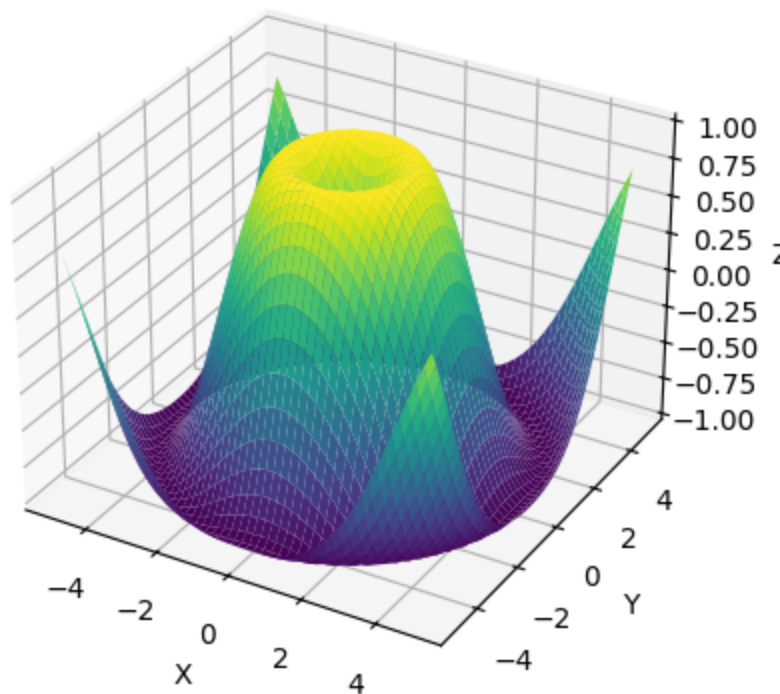
# сетка аргументов
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)

Z = np.sin(np.sqrt(X**2 + Y**2))

surf = ax.plot_surface(X, Y, Z, cmap='viridis')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Пример 3D-поверхности')

plt.show()
```

## Пример 3D-поверхности



## Построение 3D-набора точек (scatter)

Трёхмерная диаграмма рассеяния используется для визуализации набора точек в пространстве.

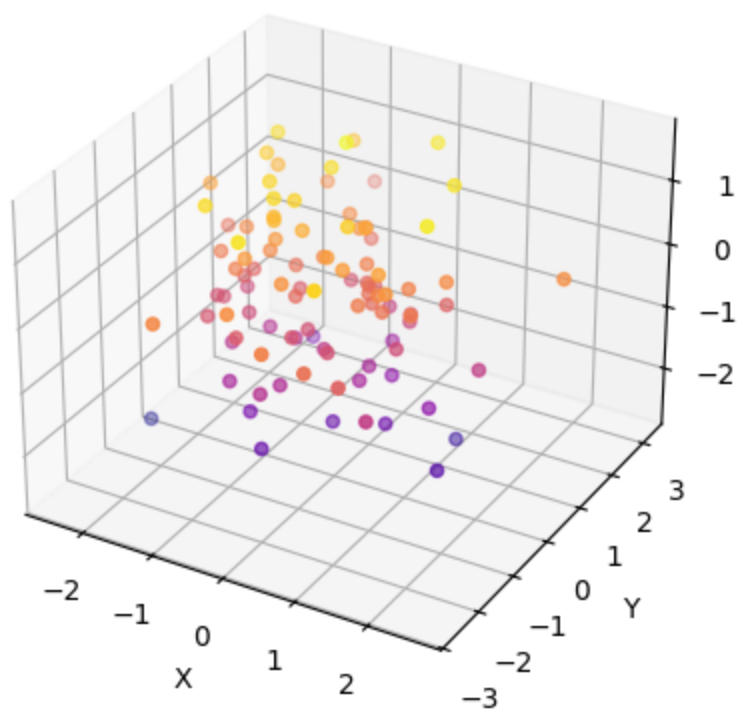
```
In [9]: fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

x = np.random.randn(100)
y = np.random.randn(100)
z = np.random.randn(100)

sc = ax.scatter3D(x, y, z, c=z, cmap='plasma')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D-диаграмма рассеяния')

plt.show()
```

### 3D-диаграмма рассеяния



## Пример 2. Сверление слоя металла лазером

Если энергия лазера полностью идет на испарение столбика металла массы  $L\rho$  ( $S$  – облучаемая площадь,  $L$  – высота столбика,  $\rho$  – плотность вещества), то закон сохранения энергии выражается равенством

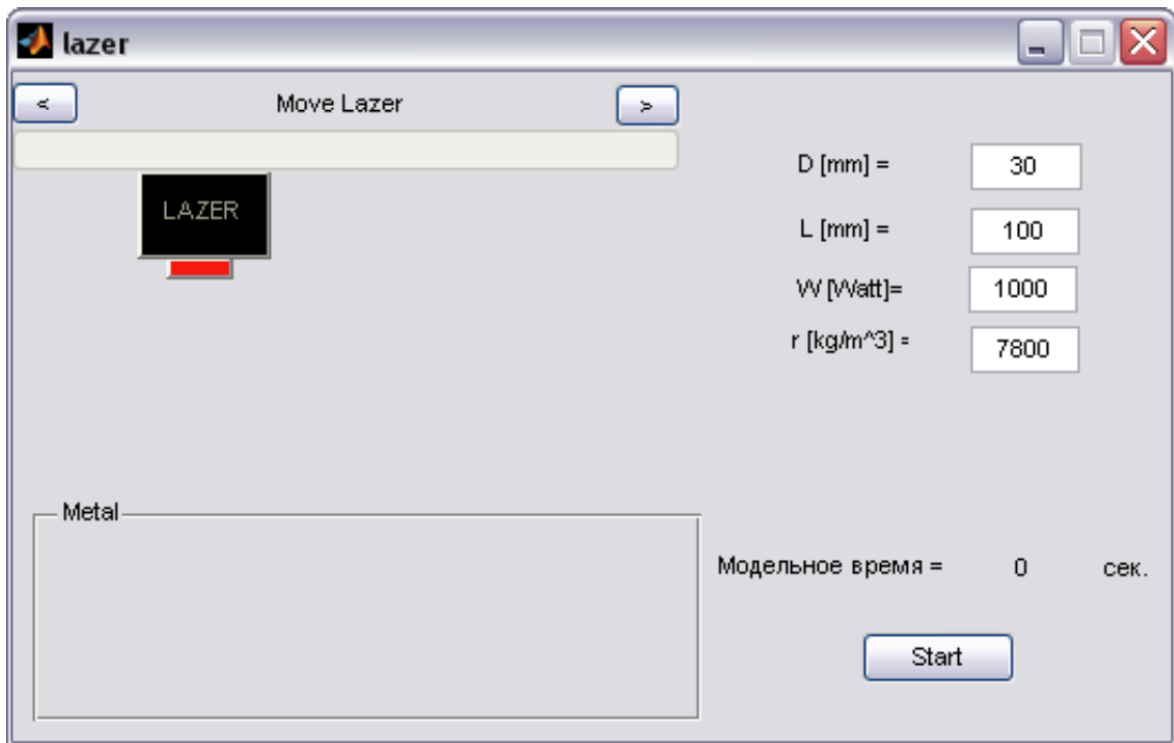
$$E_0 = W \cdot t_k = hLS\rho,$$

где  $h$  – энергия, требуемая для испарения единицы массы.

Величина  $h$  имеет составную структуру:  $h = (T_{\text{пл}} - T) \cdot h_1 + h_2 + h_3$ , поскольку материал необходимо последовательно нагреть до температуры плавления  $T_{\text{пл}}$ , а затем расплавить и превратить в пар

( $T$  – исходная температура,  $h_1$  – удельная теплоемкость,  $h_2$  и  $h_3$  – соответственно удельная теплота плавления и парообразования).

**Ниже можно наблюдать пример разработанной модели с применением GUI**



Изменение глубины выемки  $l(t)$  со временем определяется из детального баланса энергии в промежутке времени от  $t$  до  $t + dt$ . На испаренную за это время массу

$$[l(t + dt) - l(t)] \cdot S \cdot \rho = dlS\rho$$

тратится энергия  $dlhS\rho$ , равная энергии  $Wdt$ , сообщаемой веществу лазером:

$$dlhS\rho = Wdt$$

откуда получается дифференциальное уравнение

$$\frac{dl}{dt} = \frac{W}{hS\rho}$$

Его интегрирование (с учетом того, что начальная глубина выемки равна нулю) дает

$$l(t) = \frac{W}{hS\rho} \cdot t = \frac{E(t)}{hS\rho},$$

где  $E(t)$  – вся энергия, выделенная лазером к моменту времени  $t$ . Следовательно глубина выемки пропорциональная затраченной энергии.

В действительности процесс сверления гораздо сложнее рассмотренной схемы – энергия тратится на нагрев вещества, на удаление паров из выемки, которая может иметь неправильную форму, и т.д. Однако для

первоначальной оценки времени сверления отверстия модель вполне подходит.

## Средства разработки моделей с применением GUI

Для разработки GUI предлагается использование распространенной библиотеки `tkinter`.

### Создание окна приложения

```
In [ ]: import tkinter as tk

# создаём главное окно приложения
root = tk.Tk()
root.title("Пример GUI") # заголовок

# запускаем главный цикл обработки событий
root.mainloop()
```

### Создание основных элементов управления

```
In [16]: import tkinter as tk

root = tk.Tk()
root.title("Элементы управления")

# текстовая метка
label = tk.Label(root, text="Введите значение:")
label.pack()

# поле ввода
entry = tk.Entry(root)
entry.pack()

# кнопка
button = tk.Button(root, text="Выполнить")
button.pack()

root.mainloop()
```

### Взаимодействие с элементами управления

```
In [12]: import tkinter as tk

def on_button_click():
    value = entry.get() # считываем текст из поля ввода
    print("Пользователь ввёл:", value)
```

```

root = tk.Tk()
root.title("Обработчик нажатия")

entry = tk.Entry(root)
entry.pack()

button = tk.Button(root, text="Показать значение", command=on_button_click)
button.pack()

root.mainloop()

```

*Взаимодействие с элементами выбора ( Radiobutton )*

```

In [18]: import tkinter as tk

root = tk.Tk()
root.title("Выбор функции")

selected_func = tk.StringVar(value="sin") # общая переменная

rb_sin = tk.Radiobutton(root, text="sin(x)", variable=selected_func, value="sin")
rb_cos = tk.Radiobutton(root, text="cos(x)", variable=selected_func, value="cos")
rb_tan = tk.Radiobutton(root, text="tan(x)", variable=selected_func, value="tan")

rb_sin.pack(anchor="w")
rb_cos.pack(anchor="w")
rb_tan.pack(anchor="w")

def on_click():
    func_name = selected_func.get() # <-- здесь читаем выбор
    print("Выбрана функция:", func_name)

btn = tk.Button(root, text="Проверить выбор", command=on_click)
btn.pack()

root.mainloop()

```

Выбрана функция: cos

*Встраивание графика Matplotlib в окно*

Чтобы рисовать график не в отдельном окне, а внутри GUI, используют специальный холст для Matplotlib через `FigureCanvasTkAgg` и `Figure`.

```

In [14]: import tkinter as tk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.figure import Figure
import numpy as np

root = tk.Tk()
root.title("Пример встраивания графика")

# создаём объект Figure и ось

```

```

fig = Figure(figsize=(5, 4), dpi=100)
ax = fig.add_subplot(111)

# пример данных
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)
ax.plot(x, y)
ax.set_xlabel("x")
ax.set_ylabel("y")

# создаём холст для Figure внутри Tkinter
canvas = FigureCanvasTkAgg(fig, master=root)
canvas_widget = canvas.get_tk_widget()
canvas_widget.pack(fill=tk.BOTH, expand=True)

root.mainloop()

```

### Подсказка

Для обновления графика по действию пользователя, достаточно создать функцию и встроить её в действия по элементу.

```

In [ ]: import tkinter as tk

def on_plot_button_click():
    # 1) прочитать выбор функции и параметры
    # 2) сформировать массив x (например, np.linspace)
    # 3) по выбору функции вычислить y
    # 4) ax.clear(), ax.plot(x, y), подписать оси/заголовок
    # 5) canvas.draw()

button = tk.Button(root, text="Показать значение", command=on_button_click)

# ...

```

## Практическая часть

### Задание 1

Построить модель, описанную в примере 1. Получить трехмерные графики зависимостей изменения угла отклонения груза, от различных параметров.

Номер варианта вычисляется как остаток от деления порядкового номера в группе на 5  $(N_{\text{вариант}} = N_{\text{групп}} \bmod 5)$

| № варианта | Параметр   |
|------------|------------|
| 0          | Масса пули |

| № варианта | Параметр |
|------------|----------|
|------------|----------|

|   |                              |
|---|------------------------------|
| 1 | Масса груза                  |
| 2 | Длина подвеса                |
| 3 | Начальная скорость пули      |
| 4 | Ускорение свободного падения |

In [ ]: `# ваш код`

## Задание 2

Разработайте программу с GUI, которая позволяет пользователю выбрать одну из тригонометрических функций и построить ее график.

In [ ]: `# ваш код`

## Задание 3

Разработайте модель с применением GUI по **примеру 2**.

Исходными данными для модели являются:

- диаметр отверстия (мм);
- толщина металла (мм);
- мощность лазера (Вт);
- плотность металла (кг/м3).

In [ ]: `# ваш код`